

Training Notes

Computer Architecture: Computer Architecture is a blueprint and functional design of a computer system that dictates how hardware and software components interact, communicate, and execute instructions to process data.

Computer Hardware: computer Hardware refers to the physical parts of a computer that can be touched, seen, and used to execute physical operations like input, computation and storage.

- **Central Processing Unit:** known as brain of the computer. It interprets program instructions, performs arithmetic and logic operations.
- **Memory(RAM and ROM):** RAM provides volatile ,temporary working storage for active programs and Rom contains persistent bootup instructions.
- **Storage Devices:** Non volatile mediums like HDDs and solid-state drives used for long term and permanent data.

Computer Software : consists of intangible programs, data, and instructions that tell the underlying hardware what tasks to perform and how to function, Software transforms the idle physical parts into a usable computing tool.

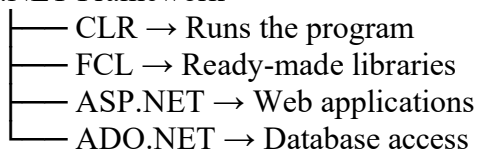
- **System Software:** programs designed to run hardware and provide a platform for other programs including operating System.
- **Application Software:** end user programs created to solve specific problems or perform specific tasks such as word processors, web browsers and media players.
- **Programing Software:** Tools used by the developers such as compilers, debuggers and interpreters – to write and translate high level code to a machine understandable code.

Microsoft .NET Framework

- .NET Framework is a Microsoft software platform for building and running applications.
- Supports languages like C#, VB.NET, and F#.
- Used for Windows, web, and database applications.
- CLR (Common Language Runtime) executes .NET programs and manages memory, exceptions, and security.
- FCL (Framework Class Library) provides ready-made classes and APIs.
- Supports OOP, garbage collection, exception handling, and code reusability.

Main Components:

.NET Framework



Microsoft Visual Studio .NET 2019

Microsoft Visual Studio 2019 is an **IDE (Integrated Development Environment)** used to develop, debug, and test .NET applications.

Main Uses

- Write **C#, VB.NET, and F#** code.
- Create **Console, Windows Forms, WPF, ASP.NET, and other applications**.
- Build and compile applications.
- Debug programs and find errors.
- Manage projects and solutions.
- Connect applications with databases such as **SQL Server**.
- Use **NuGet** packages and .NET libraries.

Terms:

Solution → Container that can hold one or more projects.

Project → Contains source code, files, references, and configuration for an application.

Editor → Area where you write code.

Solution Explorer → Displays projects and files.

Properties Window → Shows properties of selected objects.

Build → Compiles the project and checks for errors.

Debug → Runs the application while allowing you to inspect and troubleshoot the code.

NuGet → Package manager used to install external .NET libraries.

Programming Fundamentals and OOPS Concepts

Programming means writing instructions that tell a computer what to do.

A **variable** is a named memory location used to store a value.

Type	Example	Used for
int	10	Whole numbers
long	100000	Large whole numbers
float	10.5f	Decimal numbers
double	10.5	Decimal numbers
decimal	10.50m	Financial values
char	'A'	Single character
string	"Hello"	Text
bool	true	True/false

A constant is a value that cannot be changed after declaration.

e.g. `const double PI = 3.14159;`

1. Operators

Operators are symbols used to perform operations on values.

Type	Operators	Purpose
Arithmetic	+ - * / %	Mathematical operations
Relational	== != > < >= <=	Compare values
Logical	&& !	Combine conditions
Assignment	= += -= *= /=	Assign/update values
Increment/Decrement	++ --	Increase/decrease by 1
Conditional (Ternary)	? :	Short if-else
Null operators	?? ??=	Handle null values

2. Conditions

Conditions allow a program to make decisions.

Condition	Purpose
if	Executes code when a condition is true
if-else	Chooses between two options
else-if	Checks multiple conditions
switch	Selects one option from multiple fixed values
Ternary ? :	Short form of if-else

3. Loops

Loops repeat a block of code.

For : Best when the number of iterations is known.

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

While : Best when repetition depends on a condition.

```
while (count < 5)
{
    count++;
}
```

do-while : Executes at least once.

```
do
{
    count++;
} while (count < 5);
```

Foreach : Best for going through collections/arrays.

```
foreach (int n in numbers)
{
    Console.WriteLine(n);
}
```

4. Arrays

An array stores **multiple values of the same data type**.

```
int[] numbers = { 10, 20, 30, 40 };
```

Array indexing starts from **0**:

```
Index: 0  1  2  3
Value: 10 20 30 40
```

Access:

```
Console.WriteLine(numbers[0]); // 10
```

Create with fixed size:

```
int[] numbers = new int[5];
```

Get length:

```
Console.WriteLine(numbers.Length);
```

Loop through an array:

```
foreach (int number in numbers)
{
    Console.WriteLine(number);
}
```

5. Strings: String is a sequence of characters used to store text. Declared using the string keyword.

```
string name = "Bhavana";
```

Common Operations:

```
name.Length;          // Length
```

```
name.ToUpper();       // Uppercase
```

```
name.ToLower();       // Lowercase
```

```
name.Contains("a");    // Search
```

```
name.Replace("a", "b"); // Replace
```

```
name.Trim();           // Remove extra spaces
```

6. Methods

A method is a **reusable block of code** that performs a specific task.

Basic method

```
static void Greet()
{
    Console.WriteLine("Hello");
}
```

Call:

```
Greet();
```

Method with parameters

```
static void Greet(string name)
{
    Console.WriteLine("Hello " + name);
}
```

```
Greet("Bhavana");
```

Method with return value

```
static int Add(int a, int b)
{
    return a + b;
}
```

```
int result = Add(10, 20);
```

Method structure

access modifier → return type → method name → parameters

Example:

```
public int Add(int a, int b)
```

- public → access modifier
- int → return type
- Add → method name
- a, b → parameters

void

void means the method **does not return a value**.

```
static void Display()
{
```

```
Console.WriteLine("Hello");  
}
```

OOP Concepts

Class: A **class** is a blueprint for creating objects.

```
class Employee  
{  
    public string Name;  
    public int Age;  
}
```

Object: An **object** is an instance of a class.

```
Employee emp = new Employee();  
emp.Name = "Bhavana";  
emp.Age = 21;
```

Constructor: A constructor is executed when an object is created.

```
class Employee  
{  
    public string Name;  
    public Employee(string name)  
    {  
        Name = name;  
    }  
}  
  
Employee emp = new Employee("Bhavana");//object creation
```

Encapsulation: **Encapsulation** means keeping data and the operations that work on that data together, while controlling access to the data.

```
class Employee
{
    private double salary;

    public double Salary
    {
        get { return salary; }

        set { salary = value; }
    }
}
```

A common modern C# form is:

```
class Employee
{
    public string Name { get; set; }

    public decimal Salary { get; set; }
}
```

Access Modifiers: Access modifiers control who can access a class/member.

Modifier	Meaning
public	Accessible from anywhere
private	Accessible only inside the class
protected	Class + derived classes
internal	Within the same assembly

Example:

```
class Employee
{
    private decimal salary;
```

```
public string Name;  
  
}
```

Inheritance: Inheritance allows one class to inherit members from another class.

```
class Employee  
{  
    public string Name;  
    public void Work()  
    {  
        Console.WriteLine("Working");  
    }  
}  
  
class Manager : Employee  
{  
    public void Manage()  
    {  
        Console.WriteLine("Managing");  
    }  
}
```

E.g.

```
Manager manager = new Manager();
```

```
manager.Name = "John";
```

```
manager.Work();
```

```
manager.Manage();
```

Polymorphism: One interface/name can represent different behaviors.

Method Overloading : Same method name but different parameters.

Example:

```
class Calculator
```

```
{  
  
    public int Add(int a, int b)  
  
    {  
  
        return a + b;  
  
    }  
  
    public int Add(int a, int b, int c)  
  
    {  
  
        return a + b + c;  
  
    }  
  
}
```

Method Overriding: A derived class changes the behavior of a base class method.

```
class Animal  
{  
    public virtual void Sound()  
    {  
        Console.WriteLine("Animal sound");  
    }  
}  
  
class Dog : Animal  
{  
    public override void Sound()  
    {  
        Console.WriteLine("Bark");  
    }  
}  
  
Animal animal = new Dog();
```

```
animal.Sound();
```

Abstraction : Abstraction means exposing only the necessary details and hiding implementation details.

In C#, abstraction can be achieved using:

- **Abstract classes**
- **Interfaces**

```
abstract class Payment
{
    public abstract void Pay();
}
```

A derived class provides the implementation:

```
class CreditCardPayment : Payment
{
    public override void Pay()
    {
        Console.WriteLine("Payment using credit card");
    }
}
```

Interface: An interface defines a contract.

```
interface IPayment
{
    void Pay();
}
```

A class implements it:

```
class CreditCardPayment : IPayment
{
    public void Pay()
```

```
{  
  
    Console.WriteLine("Credit card payment");  
  
}  
  
}
```

Properties : A **property** is a class member used to **read, write, or control access to data**. It provides a safer and cleaner way to access private fields.

Syntax

```
public dataType PropertyName  
  
{  
  
    get { return field; }  
  
    set { field = value; }  
  
}
```

Example:

```
class Student  
  
{  
  
    private string name;  
  
    public string Name  
  
    {  
  
        get { return name; }  
  
        set { name = value; }  
  
    }  
  
}  
  
//usage  
  
Student s = new Student();  
  
s.Name = "Bhavana";    // set  
  
Console.WriteLine(s.Name); // get
```

Types of Properties

Read-write:

```
public string Name { get; set; }
```

Read-only:

```
public string Name { get; }
```

Write-only properties are possible in older patterns but are uncommon; modern C# generally uses methods or other designs instead.

Private setter:

```
public int Id { get; private set; }
```

The property can be read publicly but changed only inside the class.

Indexers : An **indexer** allows an object of a class to be accessed using an **index** (`[]`), similar to an array.

Syntax:

```
public dataType this[int index]
{
    get { }
    set { }
}
```

Example:

```
class Students
{
    private string[] names = { "A", "B", "C" };

    public string this[int index]
    {
        get { return names[index]; }
    }
}
```

```
        set { names[index] = value; }  
  
    }  
  
}  
  
Students students = new Students();  
  
Console.WriteLine(students[0]); // A  
  
students[0] = "Bhavana";  
  
Console.WriteLine(students[0]); // Bhavana
```

Exception Handling:

An **exception** is an unexpected error that occurs while a program is running.

Examples:

- Dividing by zero
- Accessing an invalid array index
- File not found
- Invalid input
- Database connection failure

Exception handling prevents the application from **crashing unexpectedly** and allows us to handle errors properly.

Keywords Used

C# provides four main keywords:

try
catch
finally
throw

try

Contains code that may cause an exception.

catch

Handles the exception.

finally

Contains code that executes whether an exception occurs or not.

throw

Used to manually generate/propagate an exception.

Example:

```
try
{
    int[] numbers = { 10, 20, 30 };
    Console.WriteLine(numbers[5]);
}
catch (IndexOutOfRangeException)
{
    Console.WriteLine("Invalid array index.");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
```

Collection Classes:

Collection classes are used to **store and manage groups of objects/data**.

Unlike arrays, many collection classes can **dynamically grow or shrink**.

Collection	Purpose
List<T>	Ordered, dynamic collection
Dictionary<TKey,TValue>	Stores key-value pairs
HashSet<T>	Stores unique values
Queue<T>	Follows FIFO
Stack<T>	Follows LIFO

List<T>

Stores multiple values in an ordered collection.

```
List<string> names = new List<string>();
names.Add("A");
```

```
names.Add("B");
names.Add("C");

names.Remove("B");

Access:

Console.WriteLine(names[0]);
```

Common methods:

```
Add()
Remove()
Contains()
Clear()
Count
```

4. Dictionary<TKey, TValue>

Stores data as **key-value pairs**. Used When you need to retrieve a value using a unique key

```
Dictionary<int, string> students = new();

students.Add(101, "Bhavana");
students.Add(102, "Joe");
Console.WriteLine(students[101]);

Key   Value
101 → Bhavana
102 → Joe
```

5. HashSet<T>

Stores **unique values** and does not allow duplicates. Used When duplicate values should not be stored.

```
HashSet<int> numbers = new();

numbers.Add(10);
numbers.Add(20);
numbers.Add(10);

Result:
10
20
```

6. Queue<T>

Follows **FIFO** — **First In, First Out**.

```
Queue<string> queue = new();

queue.Enqueue("A");
queue.Enqueue("B");

queue.Dequeue();
A → B → C
↑
First out
```

7. Stack<T>

Follows **LIFO** — **Last In, First Out**.

```
Stack<string> stack = new();

stack.Push("A");
stack.Push("B");

stack.Pop();

B is removed first.

B ← First out
A
```

Example: Undo operations.

Delegates: A **delegate** is a type-safe reference to a **method**. It allows a method to be passed as a parameter or called through a variable.

Syntax: delegate void MyDelegate(string message);

Example:

```
class Program
{
    static void Display(string message)
    {
        Console.WriteLine(message);
    }

    static void Main()
    {
        MyDelegate d = Display;
        d("Hello");
    }
}
```

Uses of Delegates

- Passing methods as parameters
- Callbacks
- Event handling
- Functional programming
- Used internally by LINQ and many .NET APIs

Events: An **event** is a mechanism that allows one object to **notify other objects when something happens**. Events are commonly based on delegates.

Example:

```
class Button
{
    public event Action Clicked;

    public void Click()
    {
        Clicked?.Invoke();
    }
}
```

Subscribe to the event:

```
Button button = new Button();

button.Clicked += () =>
{
    Console.WriteLine("Button clicked");
}
```

```
};
```

```
button.Click();//Button Clicked
```

.NET Remoting

.NET Remoting is a technology in the **.NET Framework** that allows objects in one application to communicate with objects in another application, even when they are running in different processes or machines.

- ☐ **Client** → Requests services from the remote object.
- ☐ **Server** → Hosts the remote object.
- ☐ **Remote Object** → Object whose methods can be called remotely.
- ☐ **Channel** → Handles communication between client and server.
- **Formatter** → Converts data into a format suitable for transmission.

Design Principles : Design principles are guidelines for writing software that is maintainable, reusable, flexible, and easy to understand.

SOLID Principles

S — Single Responsibility Principle (SRP)

A class should have one main responsibility.

ClaimService → Claim processing

EmailService → Email sending

O — Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification.

L — Liskov Substitution Principle (LSP)

A derived class should be usable wherever its base class is expected.

I — Interface Segregation Principle (ISP)

Prefer small, specific interfaces instead of one large interface.

D — Dependency Inversion Principle (DIP)

High-level code should depend on abstractions, not concrete implementations.

Controller



IClaimService



ClaimService

Design Patterns: A **design pattern** is a **reusable solution to a commonly occurring software design problem**.

It is not ready-made code; it is a recommended structure for solving a particular type of problem.

Pattern	Purpose
Singleton	Ensures one instance of a class
Factory	Creates objects without exposing creation logic
Repository	Separates data-access logic from business logic
Strategy	Allows different algorithms/behaviors to be selected
Observer	Notifies dependent objects when something changes
Adapter	Makes incompatible interfaces work together
Decorator	Adds behavior to an object without changing its class

Unit Testing: **Unit testing** is the process of testing a **small, independent unit of code**, usually a method or class, to verify that it works correctly.

Example:

```
public int Add(int a, int b)
{
    return a + b;
}
```

A unit Test checks: Add(10, 20) → Expected: 30

TDD is a development approach where tests are written **before the actual implementation code**.

1. Red

Write a test for functionality that doesn't exist yet.

```
[TestMethod]
public void Add_ShouldReturnSum()
{
    Assert.AreEqual(30, calculator.Add(10, 20));
}
```

Initially, the test fails.

2. Green

Write the minimum code needed to make the test pass.

```
public int Add(int a, int b)
```

```
{  
    return a + b;  
}
```

Now the test passes.

3. Refactor

Improve the code while keeping all tests passing.

RDBMS Fundamentals Concepts:

RDBMS (Relational Database Management System) is software used to **store, organize, manage, and retrieve data in tables**.

Examples:

- SQL Server
- MySQL
- Oracle
- PostgreSQL

SQL Server: Microsoft SQL Server is a **Relational Database Management System (RDBMS)** developed by Microsoft. It is used to **store, manage, retrieve, and secure data**.

SQL Server Database Engine

The core service that:

- Stores data
- Executes SQL queries
- Manages transactions
- Handles security and access
- Manages concurrency

SSMS — SQL Server Management Studio

SSMS is a graphical tool used to:

- Connect to SQL Server
- Create databases and tables
- Write and execute SQL queries
- Manage users and permissions
- Backup and restore databases

SQL Server Tools:

SQL Server provides different tools for **developing, managing, monitoring, and working with databases**.

1. SQL Server Management Studio (SSMS)

The most commonly used management tool.

Used to:

- Create/manage databases and tables
- Write and execute T-SQL queries
- Create stored procedures, views, and functions
- Manage users and permissions
- Backup and restore databases

2. SQL Server Configuration Manager

Used to manage SQL Server services and network configuration.

Used for:

- Starting/stopping SQL Server services
- Configuring SQL Server protocols
- Managing SQL Server network settings

3. SQL Server Profiler

A tool for **monitoring and tracing SQL Server activity**, such as queries and events.

Primarily a legacy/troubleshooting tool; **Extended Events** are generally preferred for modern monitoring.

4. SQL Server Agent

Used to **schedule and automate jobs**.

Examples:

- Scheduled database backups
- Maintenance tasks
- Automated SQL jobs

Availability depends on the SQL Server edition.

5. sqlcmd

A **command-line utility** used to connect to SQL Server and execute T-SQL commands.

Command Prompt → sqlcmd → SQL Server

6. Azure Data Studio

A cross-platform database tool from Microsoft for SQL Server and related data services. Microsoft has announced its retirement, with users directed toward **VS Code with the MSSQL extension**.

SQL Database: A database can also be created using the `CREATE DATABASE` command.

```
CREATE DATABASE HealthcareDB; //creating the database
USE HealthcareDB; //using the database
```

```
SELECT name
FROM sys.databases; //viewing current databases
```

```
DROP DATABASE HealthcareDB; //deleting a database
```

Enforcing Data Integrity:

Data integrity means keeping data **accurate, valid, consistent, and reliable** in a database. SQL Server enforces data integrity mainly through **constraints, relationships, and transactions**.

1. **NOT NULL** : Prevents a column from storing `NULL`.

```
MemberName VARCHAR(100) NOT NULL
```

2. **PRIMARY KEY** :uniquely identifies each row.

```
MemberId INT PRIMARY KEY
```

3. **FOREIGN KEY** : Maintains relationships between tables and prevents invalid references.

```
FOREIGN KEY (MemberId)
```

```
REFERENCES Members(MemberId)
```

4. **UNIQUE** : Prevents duplicate values.

```
Email VARCHAR(100) UNIQUE
```

5. **CHECK** : Ensures values satisfy a condition.

```
Age INT CHECK (Age >= 18)
```

6. **DEFAULT**: Provides a value when no value is supplied.

Status VARCHAR(20) DEFAULT 'Pending'

PARTITION BY : PARTITION BY is used with **window functions** to divide query results into groups without combining/removing individual rows.

Example:

```
SELECT
    MemberId,
    ClaimId,
    Amount,
    RANK() OVER (
        PARTITION BY MemberId
        ORDER BY Amount DESC
    ) AS ClaimRank
FROM Claims;
```

Ranking Functions

Ranking functions assign a **rank/position** to rows based on an ordering.

RANK()

Gives the same rank to tied values and **skips ranks** after a tie.

```
SELECT
    Name,
    Salary,
    RANK() OVER (ORDER BY Salary DESC) AS RankNo
FROM Employees;
```

Output:

Salary	Rank
--------	------

50000	1
-------	---

50000	1
-------	---

40000 3

30000 4

DENSE_RANK()

Same as RANK (), but **does not skip ranks**.

Salary	DenseRank
--------	-----------

50000	1
-------	---

50000	1
-------	---

40000	2
-------	---

30000	3
-------	---

ROW_NUMBER()

Assigns a **unique sequential number** to every row, even when values are tied.

Salary	RowNumber
--------	-----------

50000	1
-------	---

50000	2
-------	---

40000	3
-------	---

30000	4
-------	---

Retrieving and modifying Data:

```
CREATE TABLE Employees(  
  EmployeeId INT,  
  Name VARCHAR(100),  
  Age Int,  
  Salary Decimal(10,2)  
);  
  
EXEC sp_help 'Employees'; --describes the table  
EXEC sp_columns 'Employees'; --displays the columns  
  
select * from Employees;  
  
INSERT INTO Employees(EmployeeId, Name, Age, Salary) VALUES (1, 'Rudra', 25, 50000);  
  
INSERT INTO Employees(EmployeeId, Name, Age, Salary) VALUES (2, 'Priya', 28, 55000),  
(3, 'Arun', 30, 65000),  
(4, 'Sneha', 26, 60000),  
(5, 'Kiran', 32, 75000);  
  
delete from employees where EmployeeId=3;  
  
truncate table employees;
```

```
drop table Employees;
```

```
create table Categories(  
    category INT Identity(1,1) primary key,  
    categoryname varchar(100) not null,  
    description varchar(100),  
    IsActive Bit default 1  
);  
  
EXEC sp_rename 'dbo.Categories.category', 'CategoryId', 'column';  
  
EXEC sp_help categories;  
  
alter table categories add createdDate DATE;  
  
ALTER TABLE Categories  
ADD  
    DisplayOrder INT,  
    UpdatedDate DATE;  
  
ALTER TABLE Categories  
ALTER COLUMN Description VARCHAR(500);  
  
ALTER TABLE Categories  
ALTER COLUMN Description VARCHAR(500) NOT NULL;  
  
EXEC sp_rename  
    'dbo.Categories.category',  
    'CategoryId',  
    'COLUMN';  
  
ALTER TABLE Categories  
DROP COLUMN DisplayOrder;  
  
ALTER TABLE Products  
ADD CONSTRAINT CK_Products_Price  
CHECK (Price >= 0);  
  
EXEC sp_help 'Categories';  
select * from categories;  
  
alter table categories add isfeatured bit;  
ALTER TABLE Categories  
    ADD CONSTRAINT DF_Categories_IsFeatured  
    DEFAULT 0 FOR IsFeatured;  
alter table categories alter column description varchar(300);  
alter table products add brand varchar(100);  
exec sp_rename 'dbo.products.brand', 'brandname', 'column';  
alter table products drop column brandname;  
  
--to update the old rows to 0  
UPDATE Categories  
SET isfeatured = 0  
WHERE isfeatured IS NULL;
```

```

insert into Categories(categoryname,description) values ('electronics','it is product');
UPDATE Categories
SET
    CreatedDate = CAST(GETDATE() AS DATE),
    UpdatedDate = CAST(GETDATE() AS DATE)
WHERE CategoryId = 1;

```

JOINS : A **JOIN** is used to **combine data from two or more tables** based on a related column.

```

use companyDB;

create table Categoriesdetails(
    categoryId INT,
    category varchar(100)
);

create table productDetails(
    productId int primary key,
    productname varchar(100),
    categoryId INT
);

exec sp_rename 'dbo.productDetails.categoryId','categoryId','column';

INSERT INTO ProductDetails (ProductId, ProductName, CategoryId)
VALUES
(1, 'Laptop', 1),
(2, 'Mouse', 1),
(3, 'Keyboard', 1),
(4, 'Chair', 2),
(5, 'Desk', 2),
(6, 'Notebook', 3);

insert into Categoriesdetails (categoryId,category) values (1,'electronics'),(2,'furniture'),(3,'books');

select * from productDetails;
select * from Categoriesdetails;

select p.productname,c.category from productDetails p INNER JOIN Categoriesdetails c on p.categoryId = c.categoryId;

INSERT INTO ProductDetails (ProductId, ProductName, CategoryId)
VALUES
(7, 'lamp', 4)

update productDetails set categoryId=8 where productId=7;

delete from productDetails where productId=7;

select p.productname,c.category from productDetails p left JOIN Categoriesdetails c on p.categoryId = c.categoryId;

select p.productname,c.category from productDetails p right JOIN Categoriesdetails c on p.categoryId = c.categoryId;

insert into Categoriesdetails (categoryId,category) values (4,'software');

select p.productname,c.category from productDetails p full outer JOIN Categoriesdetails c on p.categoryId = c.categoryId;

select p.productname,c.category from productDetails p cross JOIN Categoriesdetails c ;

```

SQL Functions : A **function** in SQL Server is a reusable database object that performs an operation and **returns a value or a table**.

Types of Functions

1. **Scalar Function** → Returns a single value.

```
CREATE FUNCTION GetBonus(@Salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @Salary * 0.10;
END;
```

2. **Table-Valued Function (TVF)** → Returns a table.

```
CREATE FUNCTION GetEmployees()
RETURNS TABLE
AS
RETURN (SELECT * FROM Employees);
```

3. **Built-in Functions** → Predefined functions provided by SQL Server.

Aggregate → COUNT(), SUM(), AVG(), MIN(), MAX()
String → LEN(), UPPER(), LOWER(), SUBSTRING()
Date → GETDATE(), YEAR(), MONTH(), DAY()

Example:

```
CREATE TABLE Employees
(
    Id INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2)
);

INSERT INTO Employees VALUES
(1, 'Bhavana', 45000),
(2, 'Rahul', 55000);

CREATE FUNCTION CalculateBonus
(
    @Salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
```

```
RETURN @Salary * 0.10;
END;
GO

SELECT
    Name,
    Salary,
    dbo.CalculateBonus(Salary) AS Bonus
FROM Employees;
```

Stored Procedures

A **Stored Procedure** is a **precompiled collection of SQL statements** stored in the database and executed when needed.

```
CREATE PROCEDURE AddEmployee
    @Name VARCHAR(50),
    @Salary DECIMAL(10,2)
AS
BEGIN
    INSERT INTO Employees(Name, Salary)
    VALUES (@Name, @Salary);
END;
GO

EXEC AddEmployee 'Bhavana', 45000;
```

Triggers

A **trigger** is a special type of stored database program that **automatically executes when a specified event occurs** on a table or view.

Types of Triggers

1. AFTER Trigger

Executes **after** the INSERT, UPDATE, or DELETE operation completes.

```
CREATE TRIGGER trg_AfterInsert
ON Employees
AFTER INSERT
AS
BEGIN
    PRINT 'Employee added';
END;
```

2. INSTEAD OF Trigger

Executes **instead of** the specified operation.

```
CREATE TRIGGER trg_InsteadOfDelete
ON Employees
INSTEAD OF DELETE
AS
BEGIN
    PRINT 'Delete operation intercepted';
END;
```

3. DDL Trigger

Works with database-level events such as:

```
CREATE
ALTER
DROP
```

4. LOGON Trigger

Executes when a user establishes a SQL Server session.

```
CREATE TRIGGER trg_Product_Insert
ON Products
AFTER INSERT
AS
BEGIN
    INSERT INTO ProductAudit
    (
        ProductId,
        ActionType,
        ActionDate,
        ProductName
    )
    SELECT
        ProductId,
        'INSERT',
        GETDATE(),
        ProductName
    FROM inserted;
END;
GO

INSERT INTO Products
(ProductName, Price, Stock, Category)
VALUES
('Headphones', 2500, 20, 'Electronics');

select * from productAudit;
```

INDEXES

An **index** is a database structure used to **speed up data retrieval** from a table.

Think of it like an **index in a book**—instead of checking every page, SQL Server can find the required data faster.

Example

```
CREATE INDEX IX_Employees_Name  
ON Employees (Name);
```

Now searches using Name can be more efficient:

```
SELECT *  
FROM Employees  
WHERE Name = 'Bhavana';
```

Types of Indexes

1. Clustered Index

- Determines the **physical order of rows** in the table.
- A table can have **only one** clustered index.
- A primary key is commonly implemented as a clustered index by default, depending on the table definition and existing indexes.

```
CREATE CLUSTERED INDEX IX_Employees_Id  
ON Employees (EmployeeId);
```

2. Non-Clustered Index

- A separate structure that stores indexed values and pointers to the corresponding rows.
- A table can have **multiple** non-clustered indexes.

```
CREATE NONCLUSTERED INDEX IX_Employees_Name  
ON Employees (Name);
```

Cursors:

A **cursor** is a database object used to **process query results one row at a time**.

Normally, SQL works with a set of rows at once. A cursor is useful when you specifically need **row-by-row processing**.

Basic Flow

```
DECLARE → OPEN → FETCH → Process → CLOSE → DEALLOCATE
```

Example

```
DECLARE EmployeeCursor CURSOR FOR  
SELECT Name  
FROM Employees;
```

```
OPEN EmployeeCursor;  
  
FETCH NEXT FROM EmployeeCursor;  
  
CLOSE EmployeeCursor;  
DEALLOCATE EmployeeCursor;
```

Main Steps

- **DECLARE** → Define the cursor and query
- **OPEN** → Execute the cursor query
- **FETCH** → Retrieve rows
- **CLOSE** → Close the cursor
- **DEALLOCATE** → Release cursor resources

Transactions

A **transaction** is a group of SQL operations treated as **one logical unit of work**.

Either **all operations succeed**, or the changes can be **rolled back**.

Transaction Commands

```
BEGIN TRANSACTION  
COMMIT TRANSACTION  
ROLLBACK TRANSACTION
```

Example

```
BEGIN TRANSACTION;  
  
UPDATE Accounts  
SET Balance = Balance - 1000  
WHERE AccountId = 1;  
  
UPDATE Accounts  
SET Balance = Balance + 1000  
WHERE AccountId = 2;  
  
COMMIT TRANSACTION;
```

If an error occurs:

```
ROLLBACK TRANSACTION;
```

This cancels the changes made within the transaction that have not been committed.

Next Js

Next.js is a **React-based full-stack web framework** used to build modern web applications.

It provides features such as:

- Routing

- Server-side rendering
- Static site generation
- API/backend capabilities
- Image optimization
- Performance optimization

Basic Project Structure

```
my-app/
├── app/
│   ├── page.js
│   ├── layout.js
│   └── claims/
│       └── page.js
├── public/
├── package.json
└── next.config.js
```

Important Files

page.js → Defines a page.

layout.js → Shared UI/layout for pages.

public/ → Stores static files such as images.

package.json → Contains project dependencies and scripts.

Next.js uses file-system based routing.

```
app/
├── page.js      → /
├── claims/
│   └── page.js  → /claims
```

Navigate using:

```
<Link href="/claims">Claims</Link>
```

Props: Props are used to pass data from one component to another.

```
function User({ name }) {
  return <h1>{name}</h1>;
}
<User name="Bhavana" />
```

State stores data that can change during the lifetime of a component.

```
"use client";

import { useState } from "react";
```

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

Event Handling

Events respond to user actions.

```
<button onClick={handleClick}>
  Submit
</button>
```

Common events:

```
onClick
onChange
onSubmit
onMouseOver
```

Client and Server Components

In the App Router, components are **Server Components by default**.

Use:

```
"use client";
```

when a component needs client-side features such as:

- `useState`
- `useEffect`
- Browser APIs
- Interactive event handling

API / Backend Communication

Next.js applications can communicate with a backend such as **ASP.NET Core Web API**.

```
Next.js Frontend
  ↓
HTTP Request
  ↓
ASP.NET Core Web API
  ↓
EF Core
  ↓
SQL Server
```

Example:

```
const response = await fetch("https://localhost:7000/api/claims");
```

```
const data = await response.json();
```

Dynamic Routes

Used when the URL contains a changing value.

```
app/  
└─ claims/  
    └─ [id]/  
        └─ page.js
```

Example:

```
/claims/101  
/claims/102
```

Here, `id` represents the dynamic value.

LINQ (Language Integrated Query) is a C# feature used to **query and manipulate data** from different data sources using a common syntax.

LINQ can work with:

- Collections (`List`, arrays)
- Objects
- Databases
- XML
- Entity Framework

Basic Example

```
int[] numbers = { 10, 20, 30, 40, 50 };
```

```
var result = numbers.Where(n => n > 25);
```

```
foreach (var n in result)  
{  
    Console.WriteLine(n);  
}
```

Output:

```
30  
40  
50
```

Method	Purpose
<code>Where()</code>	Filter data
<code>Select()</code>	Select/transform data
<code>OrderBy()</code>	Sort ascending
<code>OrderByDescending()</code>	Sort descending
<code>First()</code>	Get first element
<code>FirstOrDefault()</code>	Get first or default

Single()	Get exactly one element
Any()	Check if any match exists
Count()	Count elements
Sum()	Calculate total
Average()	Calculate average
Max()	Find maximum
Min()	Find minimum

Example:

```
// Retrieve
var employees = db.Employees.ToList();

// Filter
var result = db.Employees
    .Where(e => e.Salary > 50000);

// Add
db.Employees.InsertOnSubmit(employee);
db.SubmitChanges();

// Update
employee.Salary = 60000;
db.SubmitChanges();

// Delete
db.Employees.DeleteOnSubmit(employee);
db.SubmitChanges();
```

Introduction to ASP.NET Core

ASP.NET Core is a **cross-platform, open-source web framework from Microsoft** used to build modern web applications, REST APIs, and backend services using **C# and .NET**.

Key Features

- **Cross-platform** → Windows, Linux, macOS
- **Open-source**
- **High performance**
- Supports **RESTful Web APIs**
- Built-in **Dependency Injection**
- **Middleware** pipeline for handling HTTP requests

- Supports **Entity Framework Core**
- Supports authentication and authorization
- Configuration through `appsettings.json`, environment variables, etc.
- Supports modern **async/await** programming

Main Components

1. Program.cs

Configures and starts the application.

2. Controllers

A **Controller** handles incoming HTTP requests and returns a response to the client.

It acts as a bridge between the **client/frontend** and the **business/data layers**.

Flow:

Client → Controller → Service → Database → Controller → Response

Handle HTTP requests and return responses.

```
[ApiController]
[Route("api/[controller]")]
public class ClaimsController : ControllerBase
{
}
```

3. Middleware

Components that process HTTP requests and responses.

```
Request
↓
Middleware
↓
Controller
↓
Response
```

4. Services

Contain business logic and are commonly registered using Dependency Injection.

5. Models / DTOs

Represent application data and API request/response structures.

6. EF Core

Used to interact with databases such as SQL Server.

Restful Web Services using web api

ASP.NET Web API / ASP.NET Core Web API is used to create HTTP services that can be accessed by web, mobile, or other applications.

Method	Operation	Example
GET	Retrieve data	Get claims
POST	Create data	Add claim
PUT	Update data	Update claim
PATCH	Partially update	Update claim status
DELETE	Delete data	Delete claim

Example:

```
[ApiController]

[Route("api/[controller]")]

public class ClaimsController : ControllerBase
{
    [HttpGet]

    public IActionResult GetClaims()
    {
        return Ok("Claims retrieved");
    }

    [HttpGet("{id}")]

    public IActionResult GetClaim(int id)
    {
        return Ok($"Claim {id}");
    }

    [HttpPost]

    public IActionResult AddClaim()
    {
        return Ok("Claim created");
    }
}
```

```

    }

    [HttpPut("{id}")]
    public IActionResult UpdateClaim(int id)
    {
        return Ok("Claim updated");
    }

    [HttpDelete("{id}")]
    public IActionResult DeleteClaim(int id)
    {
        return Ok("Claim deleted");
    }
}

```

Common HTTP Status Codes:

- 200 → OK
- 201 → Created
- 204 → No Content
- 400 → Bad Request
- 401 → Unauthorized
- 403 → Forbidden
- 404 → Not Found
- 500 → Internal Server Error

Returning Data

Web APIs commonly return **JSON**.

```

return Ok(new
{
    Id = 101,
    Status = "Approved"
});

```

Response:

```

{
  "id": 101,
  "status": "Approved"
}

```

Testing the API

You can test APIs using:

- **Swagger / OpenAPI**
- Postman
- Frontend applications such as Next.js

Example:

```
GET
https://localhost:7000/api/claims
```

Model Binding

Model Binding is the process of automatically taking data from an HTTP request and converting it into **C# objects, parameters, or model properties**.

Flow:

Client Request → Model Binding → C# Model/Parameter → Controller

Instead of manually reading each value, ASP.NET Core does it automatically.

Example:

Model:

```
public class Claim
{
    public int ClaimId { get; set; }
    public string Status { get; set; }
}
```

Controller:

```
[HttpPost]
public IActionResult Create(Claim claim)
{
    return Ok(claim);
}
```

Request JSON:

```
{
    "claimId": 101,
    "status": "Approved"
}
```

ASP.NET Core automatically converts the JSON into a `Claim` object.

Validation

Validation checks whether the data entered by the user is **valid, correct, and meets the required rules** before processing it.

Example:

- Claim ID should be required.
- Age should be between 18 and 100.
- Email should have a valid format.

Data Annotations

Data Annotations are attributes added to model properties to define validation rules.

Namespace:

```
using System.ComponentModel.DataAnnotations;
```

Common Data Annotations

Attribute	Purpose
[Required]	Field must have a value
[StringLength]	Limits string length
[MinLength]	Minimum string length
[MaxLength]	Maximum string length
[Range]	Defines numeric range
[EmailAddress]	Validates email format
[Phone]	Validates phone number
[Compare]	Compares two properties
[RegularExpression]	Validates using a pattern

Example

```
public class Claim
{
    [Required]
    public int ClaimId { get; set; }

    [Required]
    [StringLength(50)]
    public string PatientName { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Range(1, 1000000)]
    public decimal Amount { get; set; }
}
```

Validation in Controller

With [ApiController], ASP.NET Core automatically checks model validation.

```
[HttpPost]
public IActionResult Create(Claim claim)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);
}
```

```
        return Ok(claim);
    }
```

If the data is invalid, the API returns a **400 Bad Request** response.

MVC

MVC (Model–View–Controller) is a software design pattern used to separate an application into three main components.

Model

Model represents the application's **data and business rules**.

Example:

```
public class Claim
{
    public int ClaimId { get; set; }
    public string PatientName { get; set; }
    public decimal Amount { get; set; }
}
```

View

View represents the **user interface**.

Usually created using `.cshtml` files.

```
<h2>Claim Details</h2>
<p>Patient: @Model.PatientName</p>
<p>Amount: @Model.Amount</p>
```

Controller

Controller handles the request, communicates with the Model/Service, and selects the View.

```
public class ClaimsController : Controller
{
    public IActionResult Details()
    {
        var claim = new Claim
        {
            ClaimId = 101,
            PatientName = "John",
            Amount = 5000
        };

        return View(claim);
    }
}
```

MVC Request Flow

For example:

```

Browser
↓
/Claims/Details
↓
ClaimsController
↓
Get Claim Data
↓
Model
↓
Details.cshtml
↓
HTML Response
↓
Browser

```

Entity Framework (EF)

Entity Framework (EF) is a **Microsoft ORM (Object-Relational Mapper)** that allows .NET applications to interact with databases using **C# objects instead of writing SQL for every operation.**

ORM: Maps database tables ↔ C# classes.

Entity Framework Core

EF Core is the modern, lightweight and cross-platform version of Entity Framework.

It is commonly used with **ASP.NET Core Web API.**

Next.js -> ASP.NET Core Web API -> Service -> EF Core -> SQL Server

Component	Purpose
Entity	C# class representing a table
DbContext	Connects application with database
DbSet	Represents a database table
LINQ	Queries data using C#
Migrations	Creates/updates database schema

Entity Example

```

public class Claim
{
    public int ClaimId { get; set; }
    public string PatientName { get; set; }
    public decimal Amount { get; set; }
}

```

This can represent a **Claims** table.

DbContext

```

public class AppDbContext : DbContext
{
    public DbSet<Claim> Claims { get; set; }
}

```

```
}
```

`DbContext` manages the connection and communication with the database.

Retrieving Data

```
var claims = context.Claims.ToList();
```

Using LINQ:

```
var approvedClaims =  
    context.Claims  
        .Where(c => c.Status == "Approved")  
        .ToList();
```

EF Core translates the LINQ query into SQL.

CRUD Operations

```
// Create  
context.Claims.Add(claim);  
  
// Read  
var claims = context.Claims.ToList();  
  
// Update  
claim.Amount = 5000;  
context.Claims.Update(claim);  
  
// Delete  
context.Claims.Remove(claim);  
  
context.SaveChanges();
```

Routing

Routing is the process of matching an incoming **URL and HTTP method** to a specific **controller and action method**.

```
Client Request  
    ↓  
  Routing  
    ↓  
Controller → Action Method  
    ↓  
  Response
```

Attribute Routing

Most commonly used in ASP.NET Core Web API.

```

[ApiController]
[Route("api/[controller]")]
public class ClaimsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetClaims()
    {
        return Ok();
    }

    [HttpGet("{id}")]
    public IActionResult GetClaim(int id)
    {
        return Ok(id);
    }
}

```

URLs:

```

GET /api/claims
GET /api/claims/101

```

Here, {id} is a **route parameter**.

Route Parameters

Used to pass values through the URL.

```

[HttpGet("{id}")]
public IActionResult GetClaim(int id)
{
    return Ok(id);
}

```

Request:

```

/api/claims/101

id = 101

```

Route Constraints

Constraints restrict the type of value accepted.

```

[HttpGet("{id:int}")]
public IActionResult GetClaim(int id)
{
    return Ok(id);
}

```

Only an integer can be used:

```

/api/claims/101    ✓
/api/claims/abc    ✗

```

HTTP Method Routing

Attribute	HTTP Method
[HttpGet]	GET
[HttpPost]	POST
[HttpPut]	PUT
[HttpPatch]	PATCH
[HttpDelete]	DELETE

Conventional Routing

Commonly used with **MVC controllers and Views**:

```
{controller}/{action}/{id?}
```

Example:

```
/Claims/Details/101
```

Means:

- Claims → Controller
- Details → Action
- 101 → ID

ASP.NET CORE FIRST Application

1. Create the Project

In **Visual Studio 2019**:

```
File  
↓  
New  
↓  
Project  
↓  
ASP.NET Core Web Application  
↓  
Next
```

Enter:

Project Name: FirstWebAPI

Choose the appropriate installed **.NET Core** version.

Select:

ASP.NET Core Web API

Then click **Create**.

2. Project Structure

You will typically see:

```
FirstWebAPI
├── Controllers
│   └── WeatherForecastController.cs
├── Program.cs
├── Startup.cs
└── appsettings.json
```

In ASP.NET Core versions used with Visual Studio 2019, `Startup.cs` is commonly present. Newer .NET versions use a simplified `Program.cs` configuration model.

3. Create a Controller

Create:

```
Controllers
↓
HelloController.cs
```

Add:

```
using Microsoft.AspNetCore.Mvc;

namespace FirstWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class HelloController : ControllerBase
    {
        [HttpGet]
```

```
    public string Get()  
    {  
        return "Hello World";  
    }  
}
```

4. Run the Application

Press:

Ctrl + F5

or click the **Run** button.

You can test:

GET /api/hello

Response:

Hello World

If Swagger is enabled, you can also open the Swagger page and execute the GET endpoint.

